



JASHORE UNIVERSITY OF SCIENCE AND TECHNOLOGY

Department of Computer Science and Engineering

LAB REPORT

Course Code: CSE 3101

Course Title: Operating System and System Programming Lab

Lab Report No: 09

Lab Title: Implementation of First Fit, Best Fit, and Worst Fit Memory Allocation Algorithms

Submitted To:

Monishanker Halder,
Assistant Professor,
Dept. of Computer Science and Engineering,
Jashore University of Science and Technology

Submitted By:

Name: Adnan Zaman Niloy
Student ID: 210142
Session: 2021-2022
Year/Semester: 3rd Year, 1st Semester

Date of Submission: 30/07/ 2025

Experiment Title:

Implementation of First Fit, Best Fit, and Worst Fit Memory Allocation Algorithms

1. Objective

The main goal of this lab is to implement and compare three memory allocation strategies used in operating systems: **First Fit**, **Best Fit**, and **Worst Fit**. These algorithms help decide how to allocate memory blocks to processes to make the best use of available memory.

2. Theory

2.1 Memory Management

In operating systems, memory management is needed to allocate available memory blocks to different processes efficiently. It aims to minimize wasted memory (fragmentation) and make sure processes get the memory they need.

2.2 First Fit Algorithm

In **First Fit**, the OS scans memory blocks from the beginning and allocates the first block that is large enough for the process.

Advantage: Simple and fast.

Disadvantage: Can cause external fragmentation over time.

2.3 Best Fit Algorithm

In **Best Fit**, the OS finds the smallest available block that is large enough for the process.

Advantage: Tries to leave larger blocks free for bigger processes.

Disadvantage: More time-consuming to search all blocks.

2.4 Worst Fit Algorithm

In **Worst Fit**, the OS finds the largest available block and allocates it to the process.

Advantage: Leaves medium-sized blocks for other processes.

Disadvantage: Not always efficient; may lead to large unused holes.

3. Algorithm Description

3.1 First Fit Steps:

1. Start from the first block.
2. Allocate the first block that is big enough.
3. Update the block size after allocation.

3.2 Best Fit Steps:

1. Check all blocks.
2. Find the smallest block that fits.
3. Allocate it and update the block size.

3.2 Worst Fit Steps:

1. Check all blocks.
2. Find the biggest block.
3. Allocate it and update the block size.

4. Methodology

1. Take input for total memory blocks and their sizes.
2. Take input for total processes and their sizes.
3. Copy the initial block sizes into three arrays for each method.
4. Apply:
 - First Fit: Assign the first block that is large enough for each process.
 - Best Fit: Assign the smallest block that fits each process.
 - Worst Fit: Assign the largest block that fits each process.
5. Display allocation results for all three methods.
6. Trace the steps showing block usage after each allocation.

5. Source Code

```
#include <bits/stdc++.h>
using namespace std;

void firstFit(int blockSize[], int blocks, int processSize[], int processes) {
    int allocation[processes];
    fill(allocation, allocation + processes, -1);

    for (int i = 0; i < processes; i++) {
        for (int j = 0; j < blocks; j++) {
            if (blockSize[j] >= processSize[i]) {
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
            }
        }
    }
}
```

```

    }

    cout << "\nFirst Fit Allocation:\n";
    cout << "Process No.\tProcess Size\tBlock No.\n";
    for (int i = 0; i < processes; i++) {
        cout << i + 1 << "\t\t" << processSize[i] << "\t\t";
        if (allocation[i] != -1) cout << allocation[i] + 1;
        else cout << "Not Allocated";
        cout << endl;
    }
}

void bestFit(int blockSize[], int blocks, int processSize[], int processes) {
    int allocation[processes];
    fill(allocation, allocation + processes, -1);

    for (int i = 0; i < processes; i++) {
        int bestIdx = -1;
        for (int j = 0; j < blocks; j++) {
            if (blockSize[j] >= processSize[i]) {
                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
                    bestIdx = j;
            }
        }
        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blockSize[bestIdx] -= processSize[i];
        }
    }

    cout << "\nBest Fit Allocation:\n";
    cout << "Process No.\tProcess Size\tBlock No.\n";
    for (int i = 0; i < processes; i++) {
        cout << i + 1 << "\t\t" << processSize[i] << "\t\t";
        if (allocation[i] != -1) cout << allocation[i] + 1;
        else cout << "Not Allocated";
        cout << endl;
    }
}

void worstFit(int blockSize[], int blocks, int processSize[], int processes) {
    int allocation[processes];
    fill(allocation, allocation + processes, -1);

    for (int i = 0; i < processes; i++) {
        int worstIdx = -1;
        for (int j = 0; j < blocks; j++) {
            if (blockSize[j] >= processSize[i]) {
                if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
                    worstIdx = j;
            }
        }
    }
}

```

```

    }
}
if (worstIdx != -1) {
    allocation[i] = worstIdx;
    blockSize[worstIdx] -= processSize[i];
}
}
cout << "\nWorst Fit Allocation:\n";
cout << "Process No.\tProcess Size\tBlock No.\n";
for (int i = 0; i < processes; i++) {
    cout << i + 1 << "\t\t" << processSize[i] << "\t\t";
    if (allocation[i] != -1) cout << allocation[i] + 1;
    else cout << "Not Allocated";
    cout << endl;
}
}

int main() {
    int blocks, processes;
    cin >> blocks;
    int blockSize[blocks];
    for (int i = 0; i < blocks; i++) cin >> blockSize[i];

    cin >> processes;
    int processSize[processes];
    for (int i = 0; i < processes; i++) cin >> processSize[i];

    int blockSize1[blocks], blockSize2[blocks], blockSize3[blocks];
    copy(blockSize, blockSize + blocks, blockSize1);
    copy(blockSize, blockSize + blocks, blockSize2);
    copy(blockSize, blockSize + blocks, blockSize3);

    firstFit(blockSize1, blocks, processSize, processes);
    bestFit(blockSize2, blocks, processSize, processes);
    worstFit(blockSize3, blocks, processSize, processes);
    return 0;
}

```

6. Result and Discussion

Test Case Example:

5
100
500
200
300
600

4

212
417
112
426

Expected Result:

First Fit Allocation:

Process No.	Process Size	Block No.
1	212	2
2	417	5
3	112	2
4	426	Not Allocated

Best Fit Allocation:

Process No.	Process Size	Block No.
1	212	4
2	417	2
3	112	3
4	426	5

Worst Fit Allocation:

Process No.	Process Size	Block No.
1	212	5
2	417	2
3	112	5
4	426	Not Allocated.

6. Algorithm Trace

Input:

- Memory Blocks: [100, 500, 200, 300, 600]
- Processes: [212, 417, 112, 426]

First Fit Trace

Process	Size	Block Chosen	Block Status After Allocation
P1	212	Block 2 (500 → 288)	[100, 288, 200, 300, 600]
P2	417	Block 5 (600 → 183)	[100, 288, 200, 300, 183]
P3	112	Block 2 (288 → 176)	[100, 176, 200, 300, 183]
P4	426	Not allocated	(No block $\geq$ 426 left)

Best Fit Trace

Process	Size	Block Chosen	Block Status After Allocation
P1	212	Block 4 (300 → 88)	[100, 500, 200, 88, 600]
P2	417	Block 2 (500 → 83)	[100, 83, 200, 88, 600]
P3	112	Block 3 (200 → 88)	[100, 83, 88, 88, 600]
P4	426	Block 5 (600 → 174)	[100, 83, 88, 88, 174]

Worst Fit Trace

Process	Size	Block Chosen	Block Status After Allocation
P1	212	Block 5 (600 → 388)	[100, 500, 200, 300, 388]
P2	417	Block 2 (500 → 83)	[100, 83, 200, 300, 388]
P3	112	Block 5 (388 → 276)	[100, 83, 200, 300, 276]
P4	426	Not allocated	(No block ≥ 426 left)

7. Time and Space Complexity

- **Time Complexity:** $O(n * m)$ for each strategy, where **n** = number of processes, **m** = number of blocks.
- **Space Complexity:** $O(n + m)$

8. Conclusion

This lab successfully implemented First Fit, Best Fit, and Worst Fit memory allocation algorithms. By comparing the results, we can see how different strategies affect memory usage and fragmentation. This helps understand which method is more suitable for different workloads in real operating systems.